

Tiered Agent Memory (TAM)

Kiến trúc bộ nhớ nhận thức đa tầng
dành cho AI Agent

TAM Research Team

Ngày 4 tháng 5 năm 2026

Tóm tắt nội dung

Tài liệu này trình bày **Tiered Agent Memory (TAM)** — một kiến trúc bộ nhớ nhận thức đa tầng dành cho AI Agent. Khác với các hệ RAG truyền thống chỉ lưu và truy xuất dữ liệu, TAM mô phỏng quá trình nhận thức: Agent không chỉ biết *nhớ*, mà còn biết *bỏ qua* (lọc nhiễu), biết *ngghi ngờ* (khi có mâu thuẫn), biết *khái quát hóa* (nhìn bức tranh lớn) và biết *tự học* sau những lần vấp ngã. Kiến trúc gồm **4 tầng lưu trữ** (Working, Active, Latent, Archive), **3 cơ chế động lực học** (Activation, Decay, Reinforcement) và **7 lớp điều phối ngang** (Query Understanding, Competition, Abstraction, Reasoning, State, Scaling, Evolution). Bài viết cũng trình bày triển khai mã nguồn Python minh họa đầy đủ pipeline 12 bước end-to-end.

Mục lục

1	Mục tiêu của tài liệu	2
2	Mô hình tổng thể: 4 tầng + 3 cơ chế + 7 lớp điều phối	3
3	Nguyên tắc thiết kế cốt lõi	4
4	Tầng 1: Working Memory — không gian suy nghĩ hiện tại	6
5	Tầng 2: Active Memory — tri thức ổn định, truy cập mặc định	7
6	Tầng 3: Latent Memory — ký ức ngủ đông có điều kiện kích hoạt	8
7	Lớp bổ sung: Abstract Memory — lớp khái quát hoá để giảm token và tăng hiểu	9

8	Lớp bổ sung: Reasoning Memory và ReasoningBank	10
9	Tầng 4: Archive — lưu trữ lạnh, phục vụ truy xuất sâu	11
10	Đặc tả trạng thái của một memory	11
11	Đặc tả dòng chảy dữ liệu end-to-end	12
12	Các cơ chế vận hành nâng cấp	14
13	Quy tắc ghi mới, cập nhật, chuyển tầng và tự tiến hoá	16
14	Style Memory và context override	18
15	Tối ưu chi phí, hiệu năng và khả năng mở rộng	18
16	Các lỗi thiết kế phổ biến	19
17	Tiêu chí đánh giá một hệ thống memory thực tế	20
18	Kết luận lỗi	21
A	Sơ đồ kiến trúc tổng thể	22
B	Bảng tóm tắt so sánh	22
C	Thuật toán chính	23
D	Triển khai mã nguồn Python	24
E	Tài liệu tham khảo	25

1 Mục tiêu của tài liệu

Tài liệu này mô tả một kiến trúc bộ nhớ có thể dùng để thiết kế, đánh giá và vận hành Agent trong thực tế. Lỗi của kiến trúc vẫn là mô hình 4 tầng lưu trữ, nhưng mục tiêu của tài liệu không phải chỉ dừng ở việc “có chỗ để nhớ”, mà là làm rõ **hệ điều phối trí nhớ**: cái gì được nhớ, cái gì được bỏ qua, cái gì chỉ nên bật khi có cue, cái gì phải bị nghi ngờ, cái gì nên được khái quát hoá, và cái gì phải được chuyển thành chiến lược suy luận có thể tái sử dụng.

Nếu chỉ nhìn memory như một kho dữ liệu, hệ thống sẽ nhanh chóng gặp ba vấn đề: nhiều retrieval, phình Working Memory và tạo ra các phản hồi mang tính “split personality” khi thông tin cũ, mới, đúng, sai cùng chen vào ngữ cảnh. Vì vậy, một hệ thống memory tốt phải đồng thời thỏa bốn mục tiêu:

- **đúng ngữ cảnh**: Đưa đúng tri thức hoặc đúng chiến lược vào đúng lượt trả lời.
- **tiết kiệm**: Giữ Working Memory đủ nhỏ để không lãng phí token và độ trễ.
- **thích nghi**: Cho phép memory thay đổi theo thời gian, theo bằng chứng mới và theo kết quả thực thi.
- **kiểm soát được**: Giải thích được vì sao memory nào được chọn, memory nào bị ức chế, memory nào bị hạ tầng, memory nào bị đánh dấu lỗi thời và memory nào được nâng lên thành chiến lược chung.

Kiến trúc hiện tại đã đi đúng hướng nhờ có phân tầng, có Latent Memory, có trigger-based recall, có competition, có abstract memory và có gating theo intent. Tuy nhiên, để thực sự tiến lên mức production-ready và gần hơn với trí nhớ người, hệ cần bốn nâng cấp cốt lõi tiếp theo:

- **Reasoning Memory**: Lưu các chiến lược giải quyết vấn đề có thể khái quát hoá, thay vì chỉ lưu facts hay sự kiện.
- **Contrastive Signals**: Học không chỉ từ thành công mà còn từ các quỹ đạo thất bại và ngộ cụt.
- **MaTTS (Memory-aware Test-Time Scaling)**: Khi budget cao, hệ không chỉ retrieval sâu hơn mà còn mở rộng suy luận đa nhánh có memory dẫn đường.
- **Self-Evolving Background Worker**: Worker chạy nền không chỉ dọn dẹp memory mà còn tự chất lọc logs thành các memory chất lượng cao hơn rồi tiêm ngược vào Active Memory.

Nói ngắn gọn: hệ hiện tại đã “biết nhớ”, nhưng để thật sự mạnh hơn nó còn phải học cách “biết suy luận lại, biết học từ sai, biết scale đúng lúc và biết tự tiến hoá”.

2 Mô hình tổng thể: 4 tầng + 3 cơ chế + 7 lớp điều phối

Kiến trúc nên được hiểu như ba lớp chồng lên nhau.

Lớp thứ nhất là **bốn tầng lưu trữ**, trả lời câu hỏi *memory nằm ở đâu và mức độ sẵn sàng truy cập của nó là gì*. Bốn tầng này là:

- **Working Memory (WM):** Lớp context đang hoạt động trong request hiện tại.
- **Active Memory:** Lớp tri thức ổn định, truy cập nhanh, là nguồn retrieval mặc định.
- **Latent Memory:** Lớp ký ức ít dùng, chỉ bật khi có cue đủ mạnh.
- **Archive:** Lớp lưu trữ lạnh, phục vụ lưu vết, phân tích ngoại tuyến và khôi phục hiếm.

Lớp thứ hai là **ba cơ chế động lực học**, trả lời câu hỏi *memory nổi lên, mờ đi hay được làm mạnh lên bằng cách nào*. Ba cơ chế này là:

- **Activation:** Cơ chế làm một memory hoặc một chiến lược trở thành candidate cho lượt suy luận hiện tại.
- **Decay:** Cơ chế làm giảm xác suất được chọn của memory theo thời gian, theo loại memory và theo mức độ không sử dụng.
- **Reinforcement:** Cơ chế củng cố những memory đã được dùng và chứng minh là hữu ích.

Lớp thứ ba là **bảy lớp điều phối ngang** – tức các *control planes* – quyết định kiến trúc có thật sự production-ready và gần với trí nhớ người hay không. Bảy lớp đó là:

- **Query Understanding Plane:** Intent classification, query rewriting, cue expansion.
- **Competition Plane:** Competition, inhibition, dedup và selection giữa các memory cùng cạnh tranh.
- **Abstraction Plane:** Tạo ra abstract memory và trích xuất các nguyên lý chung.
- **Reasoning Plane:** Quản lý Reasoning Memory và ReasoningBank như kho chiến lược có thể tái sử dụng.
- **State Plane:** Quản lý xung đột, thời gian hiệu lực, độ hiện hành và chiến lược quên theo loại memory.
- **Scaling Plane:** Quản lý MaTTS, tức test-time scaling có memory dẫn đường.

- **Evolution Plane:** Quản lý contrastive signals, reflection và self-curation ở background worker.

Điểm mấu chốt là: **4 tầng chỉ là topology lưu trữ**. Muốn hệ “giống người” hơn và dùng được thật, cần thêm các lớp điều phối để nó không chỉ biết nhớ, mà còn biết chọn chiến lược, biết học từ sai, biết mở rộng suy luận khi cần và biết tự viết lại ký ức ở mức chất lượng cao hơn.

3 Nguyên tắc thiết kế cốt lõi

Nguyên tắc 1: Memory là hệ kích hoạt, không phải kho chứa

Một kho càng to không đồng nghĩa với một hệ trí nhớ càng tốt. Chất lượng của memory phải được đo bằng khả năng kích hoạt đúng tri thức hoặc đúng chiến lược vào đúng thời điểm, chứ không phải bằng số lượng bản ghi đang lưu.

Nguyên tắc 2: Truy vấn phải được diễn giải lại trước khi retrieval

Con người hiếm khi truy trí nhớ bằng nguyên văn câu vừa nghe. Não thường tự diễn giải lại câu hỏi thành các khái niệm gần kề. Vì vậy, một Agent tốt không nên retrieval trực tiếp từ query gốc, mà nên có bước *query rewriting/query expansion* để mở rộng từ bề mặt câu chữ sang các concept thật sự hữu ích cho recall.

Nguyên tắc 3: Memory phải cạnh tranh chứ không chỉ được xếp hạng

Việc lấy top- K theo điểm số là chưa đủ. Các memory liên quan phải **cạnh tranh** nhau để được kích hoạt, và memory mạnh hơn phải có khả năng **ức chế** những memory na ná hoặc dư thừa. Nếu không có competition/inhibition, hệ sẽ hay đưa vào WM nhiều memory giống nhau hoặc gần đúng nhưng sai ngữ cảnh.

Nguyên tắc 4: Hệ phải có cả atomic memory, abstract memory và reasoning memory

Atomic memory rất tốt cho việc lưu một ý cụ thể. Abstract memory rất tốt để nắm “big picture”. Nhưng để Agent thật sự học cách giải quyết tác vụ mới, cần thêm **Reasoning Memory** – tức lớp lưu các chiến lược giải bài toán có thể tái sử dụng giữa các tình huống khác nhau.

Nguyên tắc 5: Hệ phải học từ cả thành công lẫn thất bại

Nếu reinforcement chỉ thưởng cho những gì đã làm đúng, hệ sẽ bỏ lỡ một nửa bài học quan trọng. Các quỹ đạo đi vào ngõ cụt, các nhánh thử sai, các chiến lược không hiệu quả đều là **contrastive signals** rất có giá trị để rút ra nguyên lý sắc bén hơn.

Nguyên tắc 6: Thời gian, xung đột và độ hiện hành là thuộc tính bậc một

Một memory không chỉ cần đúng về nội dung; nó còn cần đúng về *khi nào*. Nếu không có temporal awareness và conflict resolution, hệ rất dễ giữ lại các trạng thái hoặc chiến lược đã hết hiệu lực.

Nguyên tắc 7: Intent quan trọng không kém domain

Domain gating là cần, nhưng chưa đủ. Một memory cá nhân về gym có thể cùng domain rộng với một câu hỏi tối ưu chế độ ăn, nhưng vẫn hoàn toàn không cần thiết nếu intent hiện tại là phân tích thuật toán. Vì vậy, memory usage phải được lọc theo **intent** chứ không chỉ theo domain.

Nguyên tắc 8: Không phải loại memory nào cũng quên giống nhau

Episodic memory nên quên nhanh hơn semantic memory; reasoning memory chỉ nên suy yếu khi chiến lược đó bị phản ví dụ hoặc không còn hiệu quả; system memory gần như không nên chịu decay theo cùng cơ chế với episodic.

Nguyên tắc 9: Budget cao phải scale suy luận, không chỉ scale retrieval

Attention budget kiểu cũ chỉ trả lời câu hỏi “mở bao nhiêu tầng trí nhớ”. MaTTS đi xa hơn: khi bài toán khó, budget cao phải kích hoạt suy luận đa nhánh, tuần tự hoặc song song, có memory dẫn đường để tránh đi mù quáng.

Nguyên tắc 10: Tự tiến hoá nên là trách nhiệm của background worker

Online path phải nhanh, gọn và an toàn. Công việc nặng như đọc toàn bộ logs trong ngày, so sánh success với failure, tổng hợp nguyên lý chung và viết lại memory chất lượng cao hơn nên được giao cho worker chạy nền.

4 Tầng 1: Working Memory — không gian suy nghĩ hiện tại

Vai trò hệ thống

Working Memory là phần context mà mô hình nhìn thấy trong lượt trả lời hiện tại. Với LLM, đây là nơi có quyền lực vận hành cao nhất: thứ gì không được đưa vào WM gần như không thể ảnh hưởng trực tiếp tới phản hồi.

WM không phải nơi “lưu” mà là nơi **compose**. Nó nên được dựng từ năm nguồn chính:

- Yêu cầu hiện tại.
- Lịch sử ngắn hạn của hội thoại.
- Các chỉ dẫn hệ thống đang có hiệu lực.
- Một tập nhỏ memory đã qua query rewrite, retrieval, competition, conflict filtering và compression.
- Các control parameters như recall confidence, strategy hints từ ReasoningBank và chế độ scale hiện tại của MaTTS.

Đặc tả tối thiểu

Một WM tốt nên tuân theo các quy tắc sau:

- **budget cap**: Phần memory chỉ chiếm một tỷ lệ nhỏ của tổng context.
- **top- K nhỏ**: Chỉ giữ rất ít candidate tốt nhất.
- **competition trước khi inject**: Không cho nhiều memory na ná cùng vào WM.
- **atomic + abstract + reasoning mix**: Khi phù hợp, ưu tiên một hỗn hợp nhỏ giữa chi tiết, khái quát và chiến lược thay vì nhiều mẫu rời rạc.
- **intent-aware gating**: Nếu intent hiện tại là kỹ thuật, tránh kéo memory cá nhân không liên quan.
- **meta-memory**: Nếu recall confidence thấp, WM phải phản ánh sự bất định đó thay vì trình bày memory như một sự thật chắc chắn.
- **budget-aware reasoning**: Nếu MaTTS đang ở chế độ cao, WM cần chứa chỗ cho các nhánh suy luận và các chiến lược dẫn đường.

Trade-off cốt lõi

Trade-off trung tâm của WM không chỉ là giữa “nhiều thông tin” và “ít token”, mà còn là giữa **chi tiết**, **khái quát** và **chiến lược**. Đưa quá nhiều atomic memory sẽ làm WM nặng và rối; đưa abstract memory quá sớm lại có thể làm mất chi tiết; đưa strategy quá tổng quát lại có thể khiến phản hồi thành “lý thuyết suông”. Vì vậy, WM nên luôn được xem như một hỗn hợp có kiểm soát giữa đại diện chi tiết, đại diện khái quát và gợi ý chiến lược.

5 Tầng 2: Active Memory — tri thức ổn định, truy cập mặc định

Vai trò hệ thống

Active Memory chứa tri thức có xác suất tái sử dụng cao: semantic memory, style profile, system memory và các Reasoning Memory đã được kiểm chứng. Đây là những gì Agent nên “mặc định nghĩ tới” trước tiên.

Tiêu chuẩn để một memory được ở Active

Một memory chỉ nên nằm ở Active nếu nó thỏa phần lớn các điều kiện sau:

- Có tính ổn định tương đối.
- Hữu ích cho nhiều lượt tương lai.
- Có domain và intent scope đủ rõ.
- Có độ tin cậy cao.
- Có thể biểu diễn gọn.
- Không chỉ là trạng thái ngắn hạn đã sắp hết hiệu lực.
- Nếu là strategy thì phải có bằng chứng rằng nó từng giúp giải một lớp vấn đề, chứ không chỉ một case đơn lẻ.

Cơ chế retrieval

Retrieval từ Active nên là mặc định, nhưng không phải *always-include*. Active chỉ cung cấp candidate. Sau đó, candidate còn phải qua intent filter, competition, conflict filter và budget filter trước khi được quyền vào WM.

Trong Active, nên tách ít nhất bốn họ memory:

- **semantic/personal memory:** Sở thích, vai trò, ràng buộc của người dùng.
- **system memory:** Policy, môi trường, năng lực tool.
- **style memory:** Sở thích về độ dài, giọng điệu, cấu trúc trả lời.
- **reasoning memory:** Các chiến lược giải quyết vấn đề đã được khái quát hoá và kiểm chứng.

Trade-off cốt lõi

Rủi ro lớn nhất của Active không chỉ là **over-personalization**, mà còn là **over-trusting old strategies**. Một reasoning strategy từng đúng trong nhiều case vẫn có thể trở nên sai khi ngữ cảnh thay đổi. Vì vậy, reasoning memory ở Active phải đi kèm validity scope và validation state rõ ràng.

6 Tầng 3: Latent Memory — ký ức ngủ đông có điều kiện kích hoạt

Vai trò hệ thống

Latent Memory lưu episodic memory, transient strategies, reasoning traces và các chi tiết ít dùng nhưng vẫn có giá trị tiềm tàng: sự cố cũ, thói quen ngắn hạn, dự án từng làm, triệu chứng từng nhắc tới, các chiến lược từng thử và cả những quỹ đạo đã đi vào ngõ cụt.

Nguyên tắc kích hoạt

Latent không nên được truy mặc định ở mọi lượt. Nó chỉ nên bật khi có cue đủ mạnh sau bước query rewriting. Các cue đó có thể đến từ:

- **keyword cue:** Trùng hay gần trùng từ khoá quan trọng.
- **semantic cue:** Truy vấn mới nằm gần một trải nghiệm cũ về mặt ngữ nghĩa.
- **explicit cue:** Người dùng trực tiếp yêu cầu nhớ lại.
- **risk cue:** Bài toán hiện tại có rủi ro cao nếu bỏ sót một memory cũ.
- **strategy cue:** Bài toán hiện tại giống một họ bài toán mà hệ từng thành công hoặc thất bại trước đó.

Two-pass retrieval như đặc tả mặc định

Một quy trình hợp lý là:

1. Pass 1: query rewritten được dùng để lấy candidate từ Active và ReasoningBank.
2. Pass 2: nếu trigger đủ mạnh, query mở rộng tiếp tục được dùng để lấy candidate từ Latent.

Điểm mới cần nhấn mạnh là: **query của Latent không nên là query gốc**. Nó nên là query đã được diễn giải lại để tăng recall. Đây là khác biệt rất lớn giữa “có latent memory” và “dùng latent memory hiệu quả”.

Trade-off cốt lõi

Latent cho phép hệ nhớ lại điều hiếm, nhưng cũng là nơi dễ bật nhầm nhất. Vì vậy, Latent luôn cần đi cùng competition, inhibition, recall confidence và contrastive filtering. Hệ không chỉ cần hỏi “memory nào liên quan”, mà còn phải hỏi “memory nào đáng để thử lại” và “ký ức thất bại nào nên dùng như cảnh báo thay vì dùng như lời khuyên”.

7 Lớp bổ sung: Abstract Memory — lớp khái quát hoá để giảm token và tăng hiểu

Vì sao cần

Một kiến trúc chỉ có atomic memory sẽ rất sạch về mặt retrieval, nhưng thiếu tầng khái quát. Khi người dùng có nhiều memory nhỏ cùng xoay quanh một chủ đề, hệ cần một đại diện mức cao để tóm được “big picture” mà không phải nạp quá nhiều chi tiết vào WM.

Ví dụ, các atomic memory như “thích gym”, “từng hỏi diet”, “hay hỏi calo” có thể được hợp nhất thành abstract memory: “User quan tâm sức khỏe và fitness.”

Bản chất kiến trúc

Abstract Memory không nhất thiết phải là “tầng lưu trữ thứ năm” độc lập với 4 tầng gốc. Cách đúng hơn là xem nó như một **lớp tổng hợp chạy ngang** trên Active và Latent. Nó được tạo ra bằng cách hợp nhất các memory liên quan và dùng chủ yếu cho:

- Context injection tiết kiệm token.
- Giữ mạch chủ đề nhất quán.
- Tăng khả năng suy luận ở mức khái quát.

- Làm đại diện trong competition khi nhiều atomic memory cùng chủ đề cạnh tranh nhau.
- Làm tiền đề để distill Reasoning Memory từ nhiều case riêng lẻ.

Trade-off cốt lõi

Abstract memory giúp giảm token và tăng coherence, nhưng nếu trừu tượng hoá quá sớm, hệ sẽ mất chi tiết quan trọng. Vì thế, abstract memory nên được xem như lớp **bổ sung**, không thay thế hoàn toàn atomic memory.

8 Lớp bổ sung: Reasoning Memory và Reasoning-Bank

Vì sao cần

Các tầng memory hiện có chủ yếu trả lời câu hỏi “Agent biết gì về user, hệ thống và quá khứ?”. Nhưng để Agent mang kỹ năng từ tác vụ cũ sang tác vụ mới, cần thêm lớp trả lời câu hỏi khác: “**Agent đã từng học được chiến lược giải bài toán nào?**”

Reasoning Memory không lưu “kiến thức” theo nghĩa fact đơn lẻ, mà lưu **Generalizable Reasoning Strategies**. Ví dụ, thay vì chỉ nhớ rằng một bug cụ thể đã được sửa ở commit nào, nó lưu nguyên lý: “Khi gặp lỗi X ở module Y, nên kiểm tra file config Z trước vì thường có lỗi bất đồng bộ.” Đây là loại memory có khả năng generalize cao hơn nhiều so với episodic memory.

Nó nằm ở đâu trong kiến trúc

Về mặt vận hành, Reasoning Memory nằm ở giao điểm giữa Active và Abstract:

- Nó thuộc **Active** vì phải truy cập nhanh khi gặp bài toán tương tự.
- Nó thuộc **Abstract** vì bản chất của nó là kết quả khái quát hoá từ nhiều experience.
- Các raw trajectories sinh ra chiến lược đó có thể nằm ở **Latent** hoặc **Archive**.

Vì vậy, một cách triển khai hợp lý là có một **ReasoningBank** như một sub-bank của Active/Abstract, nơi chứa các chiến lược đã qua validation.

Trade-off cốt lõi

Reasoning Memory rất mạnh vì nó giúp Agent không phải “phát minh lại cách nghĩ” ở mỗi task mới. Nhưng nó cũng nguy hiểm nếu các chiến lược được distill từ quá ít ví dụ

hoặc từ bối cảnh quá hẹp. Do đó, reasoning memory phải gắn với **applicability scope**, **validation status** và **contrastive evidence** thay vì chỉ có một câu tổng kết đẹp.

9 Tầng 4: Archive — lưu trữ lạnh, phục vụ truy xuất sâu

Vai trò hệ thống

Archive giữ lịch sử đầy đủ, snapshots, raw traces, successful and failed trajectories và dữ liệu phục vụ audit hoặc phân tích ngoại tuyến. Đây là nơi giải bài toán “lưu để tồn tại”, không phải nơi dành cho suy luận thường trực.

Đặc tả vận hành

Archive nên có:

- Chỉ mục tối thiểu theo thời gian, người dùng, domain, loại sự kiện và loại trajectory.
- Snapshot summary để có thể re-hydrate nhanh.
- Retention policy rõ.
- Khả năng trích xuất có chọn lọc lên Latent, Active hoặc ReasoningBank khi cần.
- Liên kết đủ rõ để worker có thể truy lại raw trajectories thành công và thất bại cho bước self-curate.

Trade-off cốt lõi

Archive mà không lập chỉ mục hoặc không có snapshot summary thì chỉ là bãi chứa log. Giá trị của Archive không nằm ở chỗ “giữ được nhiều”, mà ở chỗ **khi cần vẫn phục hồi được cái đúng với chi phí chấp nhận được**, và phục hồi được cả cái đúng lẫn cái sai để phục vụ contrastive learning.

10 Đặc tả trạng thái của một memory

Nếu muốn architecture thật sự triển khai được, mỗi memory cần có trạng thái tối thiểu đủ giàu để support retrieval, conflict resolution, temporal awareness, reasoning transfer và explainability. Một record tối thiểu nên mang các nhóm thuộc tính sau:

- **nhận dạng:** Mã định danh, tầng lưu trữ và loại memory.

- **nội dung:** Tóm tắt nội dung, nhóm abstraction nếu có, và nếu là reasoning memory thì cần có chiến lược cốt lõi cùng phạm vi áp dụng.
- **ngữ cảnh:** Domain tags, intent tags, module tags hoặc task family nếu cần.
- **nguồn và độ tin cậy:** Nguồn sinh ra memory, confidence, recall confidence và trạng thái kiểm chứng.
- **thời gian:** Thời điểm bắt đầu hiệu lực, kết thúc hiệu lực, chờ hiện hành, thời điểm tạo, lần xác nhận gần nhất và lần dùng gần nhất.
- **hành vi:** Usage count, reinforcement score, decay profile.
- **contrastive evidence:** Tham chiếu tới success cases, failure cases, nguyên nhân thất bại và reflection summary.
- **xung đột:** Conflict status, memory nào thay thế nó hoặc bị nó thay thế.

Không cần mọi memory đều đầy đủ mọi trường, nhưng nếu thiếu các nhóm thuộc tính này, hệ sẽ rất khó giải bài toán “memory nào đúng, còn hiệu lực, đáng tin, chuyển giao được và nên được chọn ngay lúc này”.

11 Đặc tả dòng chảy dữ liệu end-to-end

Một lượt xử lý production-ready nên được mô tả như pipeline gồm mười hai bước.

Bước 1: Phân tích intent, domain và độ phức tạp

Hệ thống xác định người dùng đang hỏi kiểu gì: hỏi kỹ thuật, hỏi cá nhân, hỏi hồi tưởng, hỏi ra quyết định hay hỏi đơn giản. Đây là bước đầu tiên quyết định memory nào được phép tham gia và task có cần test-time scaling hay không.

Bước 2: Thiết lập MaTTS budget

Thay vì chỉ quyết định “mở bao nhiêu tầng trí nhớ”, hệ xác định luôn mức scale của suy luận tại thời điểm chạy. Nếu truy vấn đơn giản, hệ đi theo fast path: ít retrieval, không mở nhiều nhánh. Nếu truy vấn khó, mơ hồ hoặc rủi ro cao, hệ bật **MaTTS** để cho phép suy luận tuần tự hoặc song song ở mức sâu hơn.

Bước 3: Query rewriting / query expansion

Query gốc của user được diễn giải lại thành các khái niệm truy trí nhớ tốt hơn. Đây là bước mở rộng từ bề mặt ngôn ngữ sang các concept có ích cho retrieval và cho lựa chọn chiến lược trong ReasoningBank.

Bước 4: Intent-aware gating

Trước khi tìm memory, hệ loại các họ memory không phù hợp với intent. Ví dụ, nếu intent là giải thích thuật toán, personal memory về gym không nên được tham gia cạnh tranh dù cùng domain rất rộng là “self-improvement”.

Bước 5: Retrieval nền từ Active và ReasoningBank

Query rewritten được dùng để truy Active qua vector search, metadata filter và domain filter. Đồng thời, hệ truy cả ReasoningBank để lấy các chiến lược từng thành công trong họ bài toán tương tự.

Bước 6: Phát hiện cue để mở Latent

Hệ thống kiểm tra xem có keyword cue, semantic cue, explicit cue, risk cue hay strategy cue đủ mạnh để mở Latent hay không.

Bước 7: Retrieval mở rộng từ Latent, Abstract Memory và raw traces

Nếu cue vượt ngưỡng, hệ dùng query expansion để truy Latent. Đồng thời, nếu nhiều atomic memory cùng một nhóm chủ đề xuất hiện, hệ có thể lấy thêm abstract memory đại diện cho nhóm đó. Với truy vấn rất khó, hệ cũng có thể lấy một số trace summaries từ Archive để cung cấp bối cảnh cho reflection hoặc branch guidance.

Bước 8: Competition, inhibition và conflict filtering

Candidate không chỉ được xếp hạng theo điểm số, mà còn **cạnh tranh** nhau. Memory mạnh hơn có thể ức chế memory gần trùng hoặc ít hữu ích hơn. Đồng thời, các memory mâu thuẫn phải bị đánh dấu hoặc giảm ưu tiên trước khi vào WM.

Bước 9: Xây dựng Working Memory

WM được dựng từ một hỗn hợp nhỏ của atomic memory, abstract memory, reasoning memory và control parameters như style override hoặc recall confidence. Nếu recall confidence thấp, hệ nên chuẩn bị phrasing phản ánh sự bất định thay vì khẳng định quá mức.

Bước 10: Kích hoạt MaTTS và suy luận đa nhánh

Nếu budget đủ cao, hệ không dừng ở một pass suy luận duy nhất, mà bật cơ chế **Memory-aware Test-Time Scaling**. Ở đây, Agent có thể chạy các nhánh suy luận tuần tự hoặc song song, tương tự Tree of Thoughts, nhưng không đi mù quáng: ReasoningBank cung cấp strategy hints để định hướng khám phá hiệu quả hơn.

Mối quan hệ đúng ở bước này là:

- **Memory dẫn đường cho Scale:** Chiến lược cũ giúp các nhánh mới đỡ lang thang.
- **Scale nuôi lại Memory:** Các nhánh mới tạo ra nhiều cách giải khác nhau, cả đúng lẫn sai, cung cấp dữ liệu cho contrastive learning.

Bước 11: Chọn đáp án, ghi lại thành công và thất bại

Sau khi các nhánh suy luận chạy xong, hệ chọn đáp án cuối cùng dựa trên chất lượng nội tại, consistency và confidence. Quan trọng hơn, nó không chỉ ghi lại nhánh thắng. Các nhánh thua, các ngõ cụt và nguyên nhân thất bại cũng được giữ lại như **contrastive signals**.

Bước 12: Update, reflection và lịch tự tiến hoá

Sau khi sinh xong, hệ cập nhật reinforcement cho memory hữu ích, áp decay theo loại memory, đánh dấu outdated nếu có conflict, cập nhật thời gian hiệu lực khi trạng thái thay đổi, và tạo **reflection summary** kiểu: “Chiến lược A thành công, chiến lược B thất bại vì nguyên nhân C.” Reflection này vừa phục vụ update ngắn hạn, vừa được đưa vào hàng đợi cho background worker tự chất lọc sau đó.

12 Các cơ chế vận hành nâng cấp

Activation + competition

Activation không nên chỉ là cộng điểm similarity, importance, recency và confidence. Nó cần thêm thành phần **competition**. Một memory được chọn không chỉ vì nó mạnh, mà còn vì nó thắng được các memory gần trùng hoặc cạnh tranh trực tiếp.

Competition giúp tránh hai lỗi phổ biến:

- Nhiều memory na ná cùng vào WM gây nhiễu.
- Memory gần đúng nhưng sai ngữ cảnh thắng vì similarity cao.

Decay + forgetting theo loại memory

Decay phải phụ thuộc vào loại memory. Một profile hợp lý là:

- **episodic memory:** Decay nhanh.
- **semantic memory:** Decay chậm.
- **reasoning memory:** Decay chậm nhưng nhạy với phản ví dụ và failure signals.
- **style/profile memory:** Decay trung bình và phụ thuộc xác nhận gần đây.
- **skill/system memory:** Gần như không decay theo cùng cơ chế với episodic.

Reinforcement + contrastive signals

Reinforcement không nên chỉ làm tăng điểm cho những gì “đã làm đúng”. Hệ phải học từ cả experience thành công và experience thất bại. Các quỹ đạo đi vào ngõ cụt không nên bị vứt bỏ hoàn toàn; chúng nên được giữ lại như **contrastive signals** để giúp hệ phân biệt “chiến lược nào nên thử lại” và “chiến lược nào nên tránh”.

Một reflection tốt không chỉ nói “cách A hiệu quả”, mà phải đi xa hơn: “A hiệu quả hơn B vì B thất bại khi giả định C không đúng.” Chính phần tương phản này mới tạo ra reasoning memory sắc bén.

Reasoning Memory formation

Reasoning Memory được hình thành khi hệ distill nhiều traces thành một chiến lược chung có thể chuyển giao. Điều quan trọng là nó không được sinh ra từ một case duy nhất nếu case đó chưa đủ đại diện. Một strategy chỉ nên được đẩy vào ReasoningBank khi:

- Nó đã xuất hiện hoặc được xác nhận ở nhiều tình huống.
- Phạm vi áp dụng của nó đủ rõ.
- Tồn tại contrastive evidence cho thấy vì sao nó tốt hơn các chiến lược cạnh tranh.

Conflict resolution

Đây là cơ chế bắt buộc nếu muốn tránh “split personality”. Khi memory mới xung đột với memory cũ, hệ phải có một chính sách rõ:

- Giảm confidence của memory cũ.
- Đánh dấu memory cũ là outdated hoặc superseded.

- Chỉ giữ memory cũ ở Latent/Archive nếu vẫn có giá trị lịch sử.
- Không cho cả hai cùng vào WM như hai sự thật hiện hành.
- Nếu xung đột nằm ở mức strategy, cần ghi rõ strategy nào bị phản ví dụ bởi case mới.

Temporal awareness

Recency và TTL là chưa đủ. Hệ cần phân biệt giữa **memory hiện hành** và **memory từng đúng**. Với reasoning memory, temporal awareness còn phải trả lời thêm: chiến lược này còn hợp thời không, hay chỉ đúng với phiên bản hệ thống, cấu hình hoặc môi trường cũ.

Meta-memory

Meta-memory là khả năng biết “mình không nhớ chắc” hoặc “mình có chiến lược cũ, nhưng chưa chắc còn áp dụng được”. Đây là bước khiến hệ gần với trí nhớ người hơn. Nếu recall confidence thấp, phản hồi nên dùng các khung như: “Tôi không hoàn toàn chắc, nhưng có vẻ trước đây...” thay vì trình bày ký ức như một fact chắc chắn.

MaTTS: Memory-aware Test-Time Scaling

MaTTS là nâng cấp trực tiếp của Attention Budget. Thay vì chỉ quyết định bật hay tắt latent retrieval, MaTTS quyết định **mức scale của quá trình suy luận**. Khi task khó, hệ có thể:

- Tăng số nhánh suy luận.
- Chạy tuần tự hoặc song song.
- Dùng ReasoningBank để định hướng các nhánh.
- So sánh các trajectory để sinh contrastive evidence mới.

Vì vậy, MaTTS không chỉ là chiến lược tiết kiệm token. Nó là cơ chế làm cho memory và reasoning cộng sinh với nhau.

13 Quy tắc ghi mới, cập nhật, chuyển tầng và tự tiến hoá

Ghi mới

Không phải mọi phát ngôn đều nên được lưu. Một write path tốt phải đánh giá:

- Độ rõ ràng của phát ngôn.
- Độ bền dự kiến.
- Giá trị cho các lượt tương lai.
- Domain và intent scope.
- Khả năng xung đột với state hiện có.
- Loại memory sẽ được gán nếu lưu.
- Liệu đây chỉ là một experience đơn lẻ hay đã đủ chất liệu để distill thành reasoning memory.

Reflection từ thành công và thất bại

Sau các tác vụ khó, hệ nên tạo ra reflection ngắn trả lời ba câu hỏi:

- Chiến lược nào đã thành công.
- Chiến lược nào đã thất bại.
- Thất bại đến từ giả định sai, thiếu thông tin hay áp dụng sai ngữ cảnh.

Reflection này là cầu nối giữa raw trajectories và Reasoning Memory.

Promotion

Memory nên được nâng từ Latent lên Active khi nó không chỉ được gọi lại nhiều lần, mà còn chứng minh được tính ổn định, ít mâu thuẫn và giá trị tái sử dụng cao. Với reasoning memory, promotion đòi hỏi mạnh hơn: chiến lược phải chứng minh được khả năng generalize qua nhiều task, không chỉ một case lẻ.

Demotion

Memory ở Active nên bị hạ xuống Latent khi không còn được dùng thường xuyên, không còn là ưu tiên mặc định, hoặc đã mất tính hiện hành nhưng vẫn đáng giữ làm bối cảnh lịch sử. Với reasoning memory, demotion xảy ra khi chiến lược từng hữu ích nhưng đã có nhiều phản ví dụ mới.

Archiving

Memory rất cũ, recall thấp và hầu như không còn giá trị cho suy luận thường nhật nên được chuyển sang Archive. Nhưng archiving không đồng nghĩa với xóa. Nó là chuyển từ “suy luận nóng” sang “lưu trữ lạnh”, đồng thời vẫn giữ được raw material để worker có thể tổng hợp lại khi cần.

Self-Evolving Background Worker

Đây là phần giúp kiến trúc thật sự “tiến hoá”. Worker chạy nền nên làm nhiều hơn việc dọn rác. Định kỳ, nó cần:

- Thu thập toàn bộ logs và trajectories của các tác vụ trong ngày.
- Tách các case success và failure.
- Gọi một LLM đóng vai **Synthesizer** để phân tích vì sao hôm nay Agent fail ở task này nhưng success ở task kia.
- Distill các nguyên lý chung thành higher-quality memory.
- Tiêm ngược các memory tốt nhất vào Active Memory hoặc ReasoningBank sau khi qua validation gate.

Đây chính là lúc hệ chuyển từ maintenance sang **self-curation**. Tức là không chỉ giữ memory sạch hơn, mà còn làm memory thông minh hơn theo thời gian.

14 Style Memory và context override

Style profile là một phần rất hữu ích của Active Memory, nhưng style không nên là luật cứng. Nó phải bị điều chỉnh bởi context hiện tại. Một cách tư duy hợp lý là:

$$\text{final_style} = 0.7 \text{ user_style} + 0.3 \text{ context_style}$$

Hệ số cụ thể chỉ là minh họa. Ý quan trọng là: style ưa thích của user nên được tôn trọng, nhưng khi tác vụ hiện tại là phân tích kỹ thuật, chứng minh toán học hoặc xử lý lỗi production, context style phải có quyền kéo phản hồi về hướng chặt chẽ hơn.

15 Tối ưu chi phí, hiệu năng và khả năng mở rộng

Những đòn bẩy tối ưu quan trọng nhất của hệ nâng cấp này là:

- **MaTTS thích ứng:** Chỉ scale suy luận đa nhánh khi thật sự cần.
- **query expansion có điều kiện:** Chỉ mở rộng sâu khi có dấu hiệu latent recall hoặc rủi ro bỏ sót cao.
- **competition trước injection:** Giảm số memory trùng chủ đề vào WM.
- **abstract memory:** Giảm token mà vẫn giữ “big picture”.
- **reasoning memory:** Tái sử dụng chiến lược thay vì khám phá lại từ đầu ở mọi task.

- **contrastive logging:** Biến thất bại thành dữ liệu học thay vì chi phí bị mất.
- **self-curation nền:** Chuyển công việc tổng hợp nặng sang worker, giữ online path gọn và nhanh.

Một hệ production-ready không phải là hệ lúc nào cũng “suy nghĩ sâu nhất”, mà là hệ biết khi nào nên suy nghĩ sâu, khi nào nên dùng chiến lược cũ, khi nào nên mở rộng đa nhánh và khi nào nên để worker nền tổng hợp lại bài học.

16 Các lỗi thiết kế phổ biến

Lỗi 1: Chỉ có top- K mà không có competition

Khi đó nhiều memory na ná cùng lọt vào WM, hoặc memory gần đúng nhưng sai ngữ cảnh không bị ức chế đủ mạnh.

Lỗi 2: Retrieval trực tiếp từ query gốc

Khi đó Latent Memory tồn tại nhưng recall rất yếu, vì hệ không tự diễn giải lại câu hỏi thành các khái niệm cần thiết.

Lỗi 3: Chỉ có atomic memory mà không có abstract memory

Khi đó hệ hoặc phải nạp nhiều mẫu nhỏ vào WM, hoặc mất đi khả năng nắm bắt tranh lớn của người dùng và tình huống.

Lỗi 4: Không có Reasoning Memory

Khi đó Agent chỉ biết facts và trải nghiệm cũ, nhưng không thật sự mang được “chiến lược giải quyết vấn đề” từ task này sang task khác.

Lỗi 5: Chỉ học từ thành công mà bỏ qua thất bại

Khi đó hệ liên tục thưởng cho những gì đã đúng, nhưng không bao giờ hiểu rõ vì sao các hướng suy luận khác lại sai, nên strategy learning sẽ nông và dễ lặp lại ngõ cụt.

Lỗi 6: Không có conflict resolution

Khi đó memory cũ và mới cùng song song tồn tại như hai sự thật hiện hành, dẫn tới phản hồi mâu thuẫn.

Lỗi 7: Không có temporal awareness

Khi đó hệ không phân biệt được “đang như thế này” với “từng như thế này”, hoặc “chiến lược từng hữu ích” với “chiến lược còn hợp thời”.

Lỗi 8: Budget chỉ điều khiển retrieval mà không scale reasoning

Khi đó high-budget mode chỉ tốn thêm token để tìm nhiều memory hơn, nhưng Agent vẫn suy nghĩ theo một đường duy nhất và bỏ lỡ lợi ích của test-time scaling.

Lỗi 9: Background worker chỉ dọn dẹp mà không self-curate

Khi đó hệ có thể sạch hơn theo thời gian, nhưng không thật sự thông minh hơn theo thời gian.

Lỗi 10: Không có meta-memory

Khi đó Agent hoặc tỏ ra “nhớ chắc” hoặc “không nhớ”, mà không thể biểu đạt trạng thái bất định hợp lý.

17 Tiêu chí đánh giá một hệ thống memory thực tế

Muốn đánh giá hệ thống này trong production, nên đặt các câu hỏi sau:

- Query rewriting có thực sự tăng latent recall mà không làm tăng quá nhiều false recall không.
- Competition có giảm redundancy trong WM không.
- Abstract memory có giúp giảm token mà vẫn giữ được chủ đề lớn không.
- Reasoning memory có giúp transfer chiến lược từ task cũ sang task mới không.
- Contrastive signals có thực sự làm các reflection sắc hơn không.
- Conflict resolution có ngăn memory mâu thuẫn cùng được coi là hiện hành không.
- Temporal awareness có phân biệt đúng giữa trạng thái hiện tại và trạng thái quá khứ không.
- MaTTS có giúp tăng chất lượng ở truy vấn khó mà không làm chi phí bùng nổ ở truy vấn đơn giản không.
- Background worker có tạo ra higher-quality memory thật sự hữu ích hay chỉ sinh thêm noise.

- Recall confidence có được hiệu chuẩn đủ tốt để phản hồi thể hiện bất định đúng mức không.

Một hệ tốt không chỉ trả lời hay, mà còn phải cho phép người thiết kế nhìn thấy vì sao nó chọn nhớ cái này, bỏ qua cái kia, nghi ngờ cái nọ, học từ cái sai nào và distill ra nguyên lý mới nào.

18 Kết luận lỗi

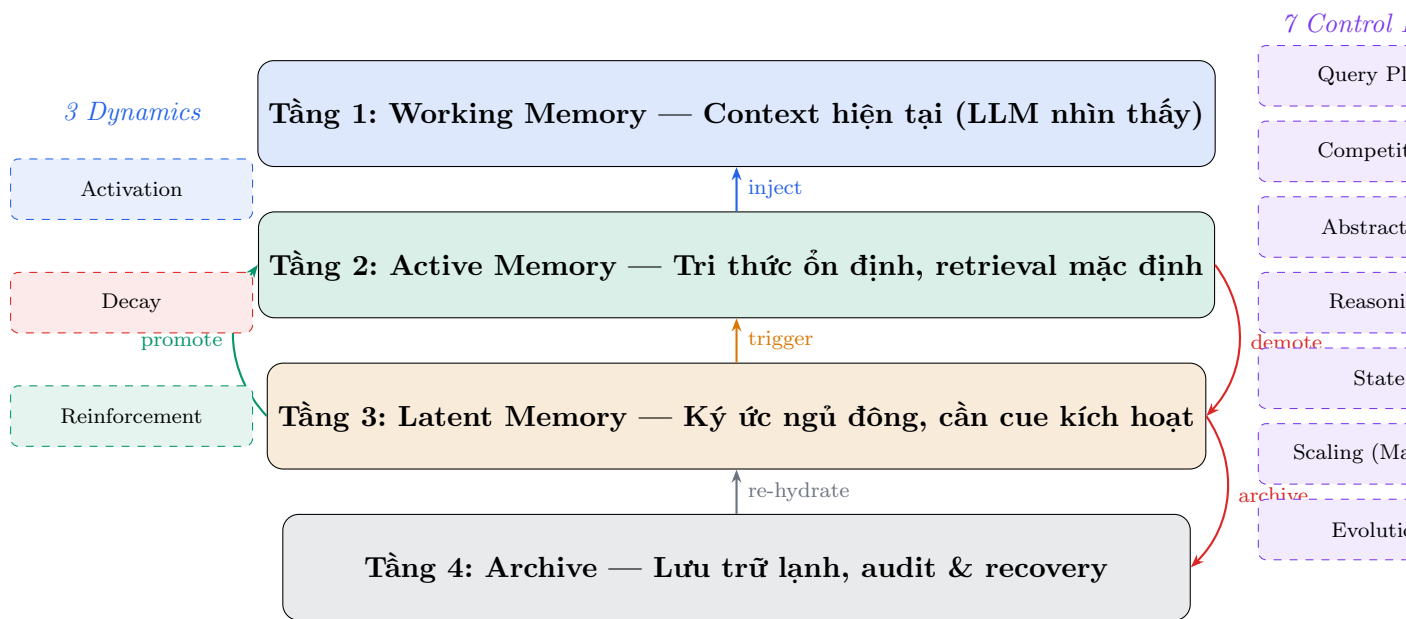
Kiến trúc 4 tầng hiện tại đã đúng hướng vì nó đã biết **phân tầng**, đã có **Latent Memory**, đã có **trigger-based recall**, đã có **competition** và đã có **abstract memory**. Đó là phần “biết nhớ” và “biết chọn” của hệ.

Nhưng để thực sự gần với trí nhớ người và đủ mạnh cho production, kiến trúc cần thực hiện bốn nâng cấp cốt lõi tiếp theo:

- **Reasoning Memory / ReasoningBank:** Để Agent không chỉ biết dữ kiện, mà còn biết chiến lược.
- **Contrastive Signals:** Để Agent học từ cả thành công lẫn thất bại.
- **MaTTS:** Để budget cao không chỉ mở thêm memory mà còn scale suy luận đa nhánh có dẫn đường.
- **Self-Evolving Background Worker:** Để hệ không chỉ được dọn dẹp, mà còn tự chất lọc và tự tiến hoá.

Nếu phải rút toàn bộ đặc tả về một câu ngắn gọn, thì câu đó là: **một hệ memory tốt không chỉ biết nhớ, mà còn biết rút chiến lược, biết học từ sai và biết tự viết lại chính trí nhớ của mình.** Chính ba năng lực đó mới làm cho kiến trúc 4 tầng đi từ mức “đúng về ý tưởng” sang mức “đủ sắc để chạy thật và đủ sống để tự tiến hoá”.

A Sơ đồ kiến trúc tổng thể



Hình 1: Kiến trúc tổng thể TAM: 4 tầng lưu trữ + 3 cơ chế động lực học + 7 lớp điều phối ngang.

B Bảng tóm tắt so sánh

So sánh 4 tầng lưu trữ

Bảng 1: So sánh đặc tính của 4 tầng lưu trữ trong TAM.				
Tầng	Tốc độ	Mặc định	Decay	Mục đích chính
Working	Rất cao	—	Không	Context đang hoạt động
Active	Cao	Có	Chậm	Tri thức ổn định, strategies
Latent	TB	Không	TB–Nhanh	Ký ức sự kiện, traces
Archive	Thấp	Không	Không	Lưu vết, audit, raw data

Decay profile theo loại memory

Bảng 2: Hệ số suy giảm (decay rate) theo loại memory — đơn vị %/ngày.

Loại Memory	Decay Rate	Nhạy với failure?	Ghi chú
Episodic	10%	Không	Quên nhanh nhất
Semantic	1%	Không	Ổn định lâu dài
Reasoning	2%	Có	Penalty khi có phản ví dụ
Style	5%	Không	Cần xác nhận định kỳ
System	0.5%	Không	Gần như không quên
Abstract	2%	Không	Tương tự semantic

C Thuật toán chính

Algorithm 1: Activation Score — Tính điểm kích hoạt cho memory candidate

Input: Memory record m , query context q , similarity s

Output: Activation score $a \in [0, 1]$

```

1 recency  $\leftarrow \exp(-0.01 \times \text{hours\_since\_last\_use}(m))$ ;
2 strategy_match  $\leftarrow \text{STRATEGYMATCH}(m.\text{type}, q.\text{intent})$ ;
3  $a \leftarrow w_1 \cdot s + w_2 \cdot m.\text{importance} + w_3 \cdot \text{recency} + w_4 \cdot m.\text{confidence} + w_5 \cdot \text{strategy\_match}$ ;
4 return  $\text{clamp}(a, 0, 1)$ ;

5 Function StrategyMatch( $type, intent$ ):
6   if  $intent = \text{technical}$  and  $type \in \{\text{reasoning}, \text{system}\}$  then
7     | return 1.0;
8   else if  $intent = \text{personal}$  and  $type \in \{\text{semantic}, \text{style}\}$  then
9     | return 1.0;
10  else if  $intent = \text{recall}$  and  $type = \text{episodic}$  then
11    | return 1.0;
12  else
13    | return 0.5;
14  end

```

Algorithm 2: Competition — MMR-style diversity selection

Input: Candidate set C , budget k , diversity λ

Output: Winner set W with $|W| \leq k$

```
1  $C \leftarrow \text{DEDUP}(C)$ ;  
2  $W \leftarrow \{\arg \max_{c \in C} \text{score}(c)\}$ ;  
3  $C \leftarrow C \setminus W$ ;  
4 while  $|W| < k$  and  $C \neq \emptyset$  do  
5   foreach  $c \in C$  do  
6      $\max\_sim \leftarrow \max_{w \in W} \text{sim}(c, w)$ ;  
7      $\text{mmr}(c) \leftarrow \lambda \cdot \text{score}(c) - (1 - \lambda) \cdot \max\_sim$ ;  
8   end  
9    $c^* \leftarrow \arg \max_{c \in C} \text{mmr}(c)$ ;  
10   $W \leftarrow W \cup \{c^*\}$ ;  $C \leftarrow C \setminus \{c^*\}$ ;  
11 end  
12 foreach  $c \in C$  (bị loại) do  
13   | Mark  $c$  as inhibited by closest winner;  
14 end  
15 return  $W$ ;
```

D Triển khai mã nguồn Python

Cấu trúc thư mục

```
tiered-agent-memory/  
  tam/  
    __init__.py          # Package init  
    config.py            # Cấu hình & siêu tham số  
    models.py            # MemoryRecord, enums  
    pipeline.py          # 12-step orchestrator  
    tiers/  
      working_memory.py  # Tầng 1 - Working Memory  
      active_memory.py   # Tầng 2 - Active (SQLite + Vector)  
      latent_memory.py   # Tầng 3 - Latent (cue-based)  
      archive.py         # Tầng 4 - Archive (cold storage)  
    dynamics/  
      activation.py       # Activation scoring  
      decay.py            # Type-aware decay  
      reinforcement.py    # Contrastive signals  
    planes/  
      query_plane.py     # Intent + query rewriting  
      competition.py      # MMR competition  
      abstraction.py      # Abstract memory synthesis  
      reasoning.py        # ReasoningBank  
      state_plane.py      # Conflict + temporal  
      scaling.py          # MaTTS  
      evolution.py        # Self-evolving worker  
    demo.py              # CLI demo  
    tests/
```

Ví dụ sử dụng

Listing 1: Khởi tạo và truy vấn TAM Pipeline

```
from tam import TAMPipeline, TAMConfig, MemoryType

# Khởi tạo
config = TAMConfig(db_path="my_agent.db")
pipeline = TAMPipeline(config)

# Thêm memory
pipeline.add_memory(
    "User thích lập trình Python",
    memory_type=MemoryType.SEMANTIC,
    domain_tags=["technical"],
)

# Truy vấn - chạy 12 bước pipeline
result = pipeline.process("Lam sao fix bug timeout?")

# Kết quả
print(result["query_context"]["intent"]) # "technical"
print(result["retrieval"]["winners"])   # So memory thang
print(result["wm_summary"])              # Trạng thái WM
```

Công nghệ sử dụng

Thành phần	Công nghệ
Persistent storage	SQLite (built-in Python)
Vector search	NumPy cosine similarity
Embedding	Hash-based (demo) / sentence-transformers (production)
Competition	MMR — Maximal Marginal Relevance
Test framework	pytest

E Tài liệu tham khảo

- [1] C. Packer et al., “MemGPT: Towards LLMs as Operating Systems,” *arXiv:2310.08560*, 2023.
- [2] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” *NeurIPS*, 2023.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” *NeurIPS*, 2023.
- [4] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.

- [5] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” *SIGIR*, 1998.
- [6] A. Baddeley, “Working Memory: Theories, Models, and Controversies,” *Annual Review of Psychology*, 2012.
- [7] E. Tulving, “Episodic and semantic memory,” *Organization of Memory*, 1972.
- [8] J. R. Anderson, “A spreading activation theory of memory,” *Journal of Verbal Learning and Verbal Behavior*, 1983.